

Preentrega de proyecto - Curso Java Back-End

Talento Tech

Introducción

La presente pre-entrega consiste en el desarrollo de un sistema de gestión de productos y pedidos realizado en Java mediante una aplicación de consola.

El objetivo principal del trabajo fue aplicar los conceptos fundamentales de programación orientada a objetos (POO), estructuras de datos, modularización del código y manejo de excepciones, utilizando una arquitectura organizada en múltiples clases con responsabilidades específicas.

El sistema permite administrar un catálogo de productos, controlar el stock disponible y gestionar pedidos realizados por los usuarios. Además, incorpora validaciones de entrada de datos y excepciones personalizadas para garantizar un funcionamiento más seguro y robusto.

Durante el desarrollo se implementaron conceptos clave del lenguaje Java, como encapsulamiento, uso de colecciones dinámicas, relaciones entre objetos, reutilización de código y separación entre lógica de negocio e interfaz de usuario.

La aplicación fue diseñada siguiendo una estructura modular, facilitando futuros avances y mejoras como persistencia en base de datos, interfaz gráfica, desarrollo de una API o la utilización de frameworks como Spring.

El sistema permite:

- Agregar productos
- Listar productos
- Buscar productos por ID
- Actualizar productos
- Eliminar productos
- Crear pedidos
- Listar pedidos
- Ver pedidos específicos
- Eliminar pedidos

El sistema desarrollado funciona completamente desde consola y permite administrar productos y pedidos de manera dinámica, validando errores y controlando el stock disponible en tiempo real.

Estructura General del Proyecto

El proyecto fue dividido en distintas clases para mantener una correcta organización y separación de responsabilidades.

Las principales clases utilizadas son:

<i>CLASE</i>	<i>FUNCIÓN</i>
Producto	Representa un producto del sistema
Pedido	Representa un pedido realizado
ItemPedido	Relaciona un producto con una cantidad
ProductoService	Maneja la lógica de productos
PedidoService	Maneja la lógica de pedidos
MenuPrincipal	Gestiona el menú principal de la App
MenuGestionPedidos	Gestiona el menú de pedidos
Validador	Valida los datos ingresados
Main	Punto de inicio del programa
Excepciones personalizadas	Manejan errores específicos

Clase Producto

La clase **Producto** representa cada artículo almacenado en el sistema.

Contiene los siguientes atributos:

- id
- nombre
- precio
- stock
- categoria
- marca

Se implementaron:

- Constructores
- Getters y setters
- Método **toString()**

El método **toString()** fue sobrescrito para mostrar la información del producto de manera ordenada y legible en consola.

Conceptos aplicados:

- Encapsulamiento
- Sobrescritura de métodos
- Uso de atributos privados

Es importante mencionar que la clase **ProductoService** es la responsable de asignar los Ids a los productos de forma dinamica al momento de guardarlos en la lista correspondiente.

```

----- Catalogo -----
ID: 1 | Redmi Note 10 | $ 250000,00 | Stock: 5 | Categoría: Celular | Marca: Xiaomi
ID: 2 | Galaxy A16 4G | $ 399500,00 | Stock: 3 | Categoría: Celular | Marca: Samsung
ID: 3 | Jd Capri 1.83 | $ 40500,00 | Stock: 3 | Categoría: Smartwatch | Marca: JD
ID: 4 | Gamer G435 | $ 126000,00 | Stock: 10 | Categoría: Auriculares | Marca: Logitech G
ID: 5 | Ecotank L1250 | $ 335699,00 | Stock: 2 | Categoría: Impresora | Marca: Epson
ID: 6 | Lenovo M11 8GB | $ 432899,00 | Stock: 5 | Categoría: Tablet | Marca: Lenovo
ID: 7 | Exo 14 Intel | $ 271999,00 | Stock: 3 | Categoría: Notebook | Marca: Exo
ID: 8 | Gamer 24 Master | $ 141300,00 | Stock: 7 | Categoría: Monitor | Marca: Cooler Master
ID: 9 | Gadnic N4020 | $ 328799,00 | Stock: 7 | Categoría: Notbook | Marca: Gadnic
ID: 10 | Acer Aspire | $1439999,00 | Stock: 2 | Categoría: Notbook | Marca: Acer
  
```

Clase ProductService

La clase **ProductoService** contiene toda la lógica relacionada con productos.

Las funcionalidades implementadas fueron:

<i>Funcionalidad</i>	<i>Detalle</i>
Guardar productos	Se agregan productos a una lista en memoria.
Listar productos	Se muestran todos los productos cargados
Buscar productos por ID	Se recorre la lista hasta encontrar el producto solicitado
Actualizar producto	Se modifican los datos de un producto existente.
Eliminar producto	Se elimina un producto de la lista.

Además, se implementó un contador automático de IDs utilizando una variable estática.

Conceptos aplicados:

- CRUD completo
- Manejo de listas
- Métodos de servicio
- Variables estáticas

Clase ItemPedido

La clase **ItemPedido** funciona como intermediaria entre **Pedido** y **Producto**.

Su función es almacenar:

- El producto seleccionado
- La cantidad comprada

Esto permite que un mismo pedido contenga varios productos diferentes.

Conceptos aplicados:

- Relaciones entre objetos
- Asociación de clases

Clase Pedido

La clase `Pedido` representa una compra realizada por un cliente.

Cada pedido contiene:

- Un ID
- Una lista de productos con sus cantidades (`List<ItemPedido>`)
- El total a pagar

La clase utiliza una lista dinámica (`ArrayList`) para almacenar los productos agregados al pedido.

El método `getTotalAPagar()` calcula automáticamente el total recorriendo todos los productos del pedido.

Conceptos aplicados:

- Colecciones (`ArrayList`)
- Composición entre clases
- Iteración con `for-each`

```
***** (ID) PEDIDO N°1 *****
Producto: Ecotank L1250 | Cantidad: 2 | Precio Unitario: 335699,00
Producto: Jd Capri 1.83 | Cantidad: 1 | Precio Unitario: 40500,00
Total a pagar $711898.0

***** (ID) PEDIDO N°2 *****
Producto: Gadníc N4020 | Cantidad: 2 | Precio Unitario: 328799,00
Producto: Lenovo M11 8GB | Cantidad: 3 | Precio Unitario: 432899,00
Producto: Exo 14 Intel | Cantidad: 1 | Precio Unitario: 271999,00
Total a pagar $2228294.0
```

Clase PedidoService

La clase `PedidoService` administra los pedidos realizados mediante una lista de pedidos (`List<Pedido>`)

Las funcionalidades implementadas fueron:

- Guardar pedidos
- Listar pedidos
- Buscar pedido por ID
- Eliminar pedidos

También se utilizó un contador automático para generar IDs únicos.

Conceptos aplicados:

- Gestión de entidades
- Búsqueda en listas
- Reutilización de lógica

Menú Principal

La clase `MenuPrincipal` administra la interacción principal con el usuario desde consola.

El menú permite:

1. Agregar producto
2. Listar productos
3. Buscar producto
4. Actualizar producto
5. Eliminar producto
6. Ingresar al módulo de pedidos

En esta parte se trabajó principalmente con:

- Entrada de datos mediante `Scanner`
- Estructuras `switch`
- Ciclos `do-while`

```
=====
==== SISTEMA DE GESTIÓN - TECHLAB ====
=====
1) Agregar producto
2) Listar productos
3) Buscar producto por Id
4) Actualizar productos
5) Eliminar producto
6) Crear pedido
0) Salir
=====
Elija una opcion: █
```

Menú Gestión de Pedidos

La clase `MenuGestionPedidos` contiene toda la lógica relacionada con pedidos.

El menú permite:

1. Crear un pedido
2. Listar pedidos
3. Ver pedido
4. Eliminar pedido

Las funcionalidades incluyen:

<i>Funcionalidad</i>	<i>Detalle</i>
Crear pedido	El usuario selecciona productos y cantidades
Validar Stock	Antes de agregar un producto al pedido, el sistema verifica si hay stock suficiente.
Actualizar stock automáticamente	Cuando se realiza un pedido el stock de los productos seleccionados disminuye.

Eliminar pedido	Si se elimina el pedido el stock vuelve a restaurarse.
-----------------	--

Esta parte fue importante porque permitió implementar lógica de negocio real.

Conceptos aplicados:

- Validaciones
- Actualización de datos
- Relaciones entre entidades

```
=====
=====  GESTIÓN DE PEDIDOS  =====
=====
1) Crear pedido
2) Listar pedidos
3) ver pedido
4) Eliminar pedido
0) Regresar al menu principal
=====
Elija una opcion: |
```

Clase Validador

La clase `Validador` fue creada para centralizar las validaciones de entrada de datos.

Incluye métodos para leer:

- Enteros
- Números decimales
- Cadenas de texto

Además, se implementó manejo de errores mediante `try-catch` para evitar que el programa finalice si el usuario ingresa datos incorrectos.

Conceptos aplicados:

- Validación de datos
- Manejo de excepciones
- Reutilización de código

Manejo de Excepciones

Se implementaron excepciones personalizadas para controlar errores específicos del sistema:

<i>Excepción</i>	<i>Función</i>
<code>ProductoNoEncontradoException</code>	Se lanza cuando no existe un producto
<code>PedidoNoEncontradoException</code>	Se lanza cuando no existe un pedido
<code>StockInsuficienteException</code>	Se lanza cuando no hay stock suficiente

Esto permitió mejorar la robustez del programa y mostrar mensajes claros al usuario.

Conceptos aplicados:

- Herencia
- RuntimeException
- Manejo de errores personalizados

Clase Main

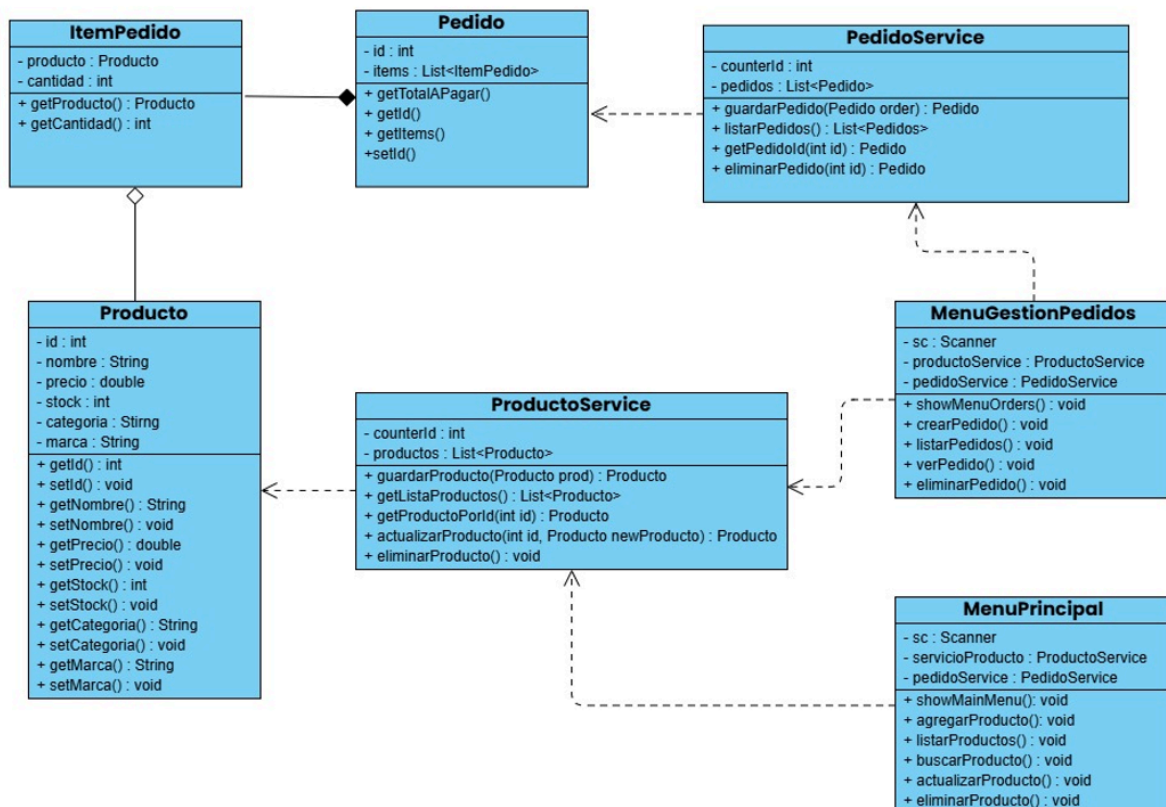
La clase **Main** es el punto de entrada del programa.

Sus responsabilidades son:

- Crear las instancias principales
- Inicializar servicios
- Mostrar los menús
- Controlar el flujo general del sistema

También se creó un método llamado **cargarProductosTest()** para precargar productos automáticamente y facilitar las pruebas del sistema.

UML (diagrama de clases)



Conclusión

Esta primera parte del proyecto permitió aplicar de manera práctica los conceptos fundamentales de Java. .

A través del desarrollo del sistema se logró:

- Organizar correctamente el código en múltiples clases
- Separar lógica de negocio y presentación
- Implementar validaciones y manejo de errores
- Simular un sistema real de gestión de productos y pedidos